Introducción a los Sistemas Operativos

Introducción – II

I.S.O.

- Versión: Agosto 2024
- Palabras Claves: Sistema Operativo, Hardware, System Call, Interacción, Modos de Ejecución, Protección, Kernel

Los temas vistos en estas diapositivas han sido mayormente extraídos del libro de William Stallings (Sistemas Operativos: Aspectos internos y principios de diseño) y Conceptos de sistemas operativos (Silberschatz, Galvin, Gagne)

Funciones principales de un SO

- Brindar abstracciones de alto nivel a los procesos de usuario
- Administrar eficientemente el uso de:
  - La CPU
  - La memoria
  - Otros dispositivos
- Brindar asistencia para la realización de Entrada-Salida (E/S) por parte de los procesos

Problemas que un SO debe evitar

- Que un proceso se apropie de la CPU
- Que un proceso intente ejecutar instrucciones de "importantes"
  - E/S / flags del procesador /
- Que un proceso intente acceder a una posición de memoria fuera de su espacio declarado.
  - Proteger los <u>espacios de direcciones</u>

Para ello, el SO entre otras debe:

- Gestionar el uso de la CPU
- Detectar intentos de ejecución de instrucciones de E/S ilegales
- Detectar accesos ilegales a memoria
- Proteger el vector de interrupciones
  - Así como las RAI (Rutinas de atención de interrupciones)

Apoyo del Hardware

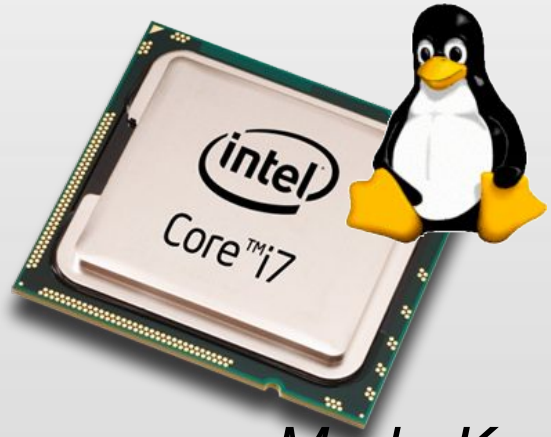
- Modos de Ejecución: Define limitaciones en el conjunto de instrucciones que se puede ejecutar en cada modo
- Interrupción de Clock: Se debe evitar que un proceso se apropie de la CPU
- Protección de la Memoria: Se deben definir límites de memoria a los que puede acceder cada proceso (registros base y límite)

Modos de ejecución

- El bit en la CPU indica el modo actual
- Las instrucciones privilegiadas deben ejecutarse en modo **Supervisor o Kernel**
  - Necesitan acceder a estructuras del kernel, o ejecutar código que no es del proceso
- En modo **Usuario**, el proceso puede acceder sólo a su espacio de direcciones, es decir a las direcciones "propias".

Modos de ejecución (cont)

- El kernel del SO se ejecuta en modo supervisor
- El resto del SO y los programas de usuario se ejecutan en modo usuario (subconjunto de instrucciones permitidas)



Modo Kernel

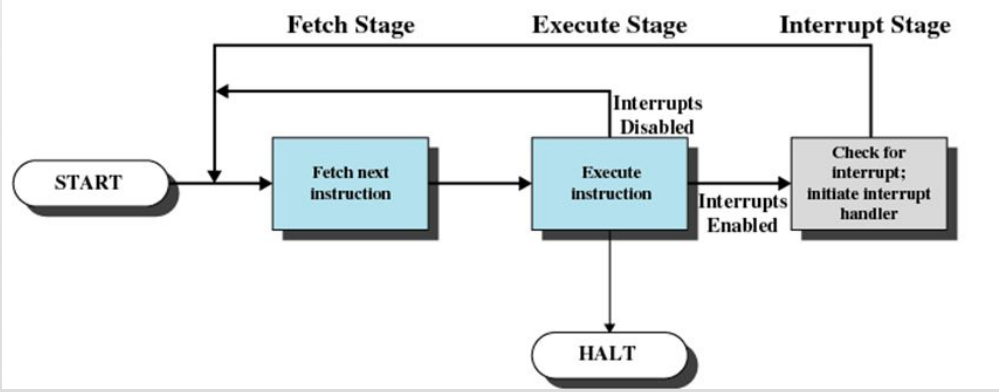
Modo Usuario

Tener en cuenta que…

- Cuando se arranque el sistema, arranca con el bit en modo supervisor.
- Cada vez que comienza a ejecutarse un proceso de usuario, este bit se DEBE PONER en modo usuario.
  - Mediante una Instrucción especial.
- Cuando hay un trap o una interrupción, el bit de modo se pone en modo Kernel.
  - <u>Única</u> forma de pasar a Modo Kernel
  - <u>No</u> es el proceso de usuario quien hace el cambio explicitamente.

Cómo actúa…

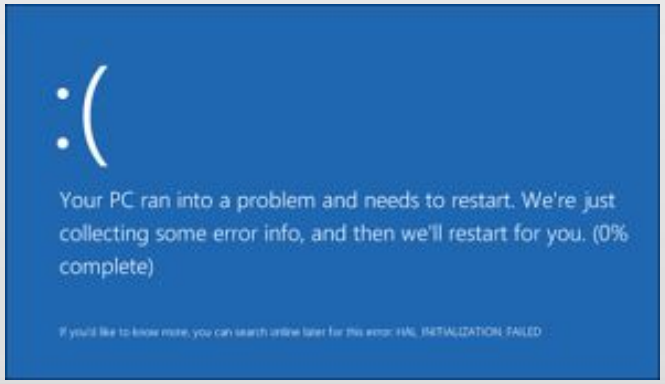
- Cuando el proceso de usuario intenta por sí mismo ejecutar instrucciones que pueden causar problemas (las llamadas instrucciones privilegiadas), el HW lo detecta como una operación ilegal y produce un trap al SO.



En Windows…

- En WIN2000 el modo núcleo ejecuta los servicios ejecutivos. El modo usuario ejecuta los procesos de usuario.

Cuando un programa se bloquea en modo usuario, a lo sumo se escribe un suceso en el registro de sucesos. Si el bloqueo se produce estando en modo supervisor
se genera la BSOD
(pantalla azul de la muerte).



Modos de Ejecución

- Procesador Intel 8088 no tenía modo dual de operación ni protección por hardware.
- En MsDos las aplicaciones pueden acceder directamente a las funciones básicas de E/S para escribir directamente en pantalla o en disco.

Resumiendo…

- **Modo kernel:**
  - **Gestión de procesos:** Creación y terminación , planificación, intercambio, sincronización y soporte para la comunicación entre procesos
  - **Gestión de memoria:** Reserva de espacio de direcciones para los procesos, Swapping, Gestión y páginas de segmentos
  - **Gestión E/S:** Gestión de *buffers, r*eserva de canales de E/S y de dispositivos de los procesos
  - **Funciones de soporte:** Gestión de interrupciones, auditoría, monitoreo
- **Modo usuario:**
  - ✔ Debug de procesos, definición de protocolos de comunicación gestión de aplicaciones (compilador, editor, aplicaciones de usuario
  - ✔ En este modo se llevan a cabo todas las tareas que no requieran accesos privilegiados
  - ✔ En este modo no se puede interactuar con el hardware
  - ✔ El proceso trabaja en su propio espacio de direcciones

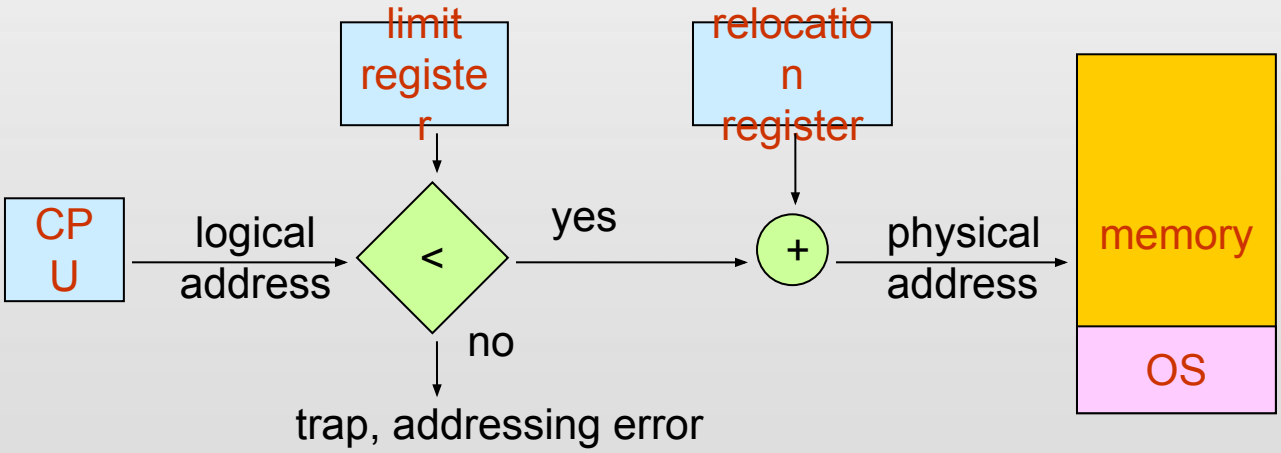
Protección de la memoria

- Delimitar el espacio de direcciones del proceso
- Poner limites a las direcciones que puede utilizar un proceso
  - Por ejemplo: Uso de un registro base y un registro límite
  - El kernel carga estos registros por medio de instrucciones privilegiadas. Esta acción sólo puede realizarse en modo Kernel

Protección de la memoria (cont)

La memoria principal aloja al SO y a los procesos de usuario

- ✔ El kernel debe proteger para que los procesos de usuario no puedan acceder donde no les corresponde
- ✔ El kernel debe proteger el espacio de direcciones de un proceso del acceso de otros procesos.



Protección de la E/S

- Las instrucciones de E/S se definen como privilegiadas.
- Deben ejecutarse en Modo Kernel
  - Se deberían gestionar en el kernel del sistema operativo
  - Los procesos de usuario realizan E/S a través de llamadas al SO (es un servicio del SO)

Protección de la CPU

- Uso de interrupción por clock para evitar que un proceso se apropie de la CPU
- Se implementa normalmente a través de un clock y un contador.
- El kernel le da valor al contador que se decrementa con cada tick de reloj y al llegar a cero puede expulsar al proceso para ejecutar otro.

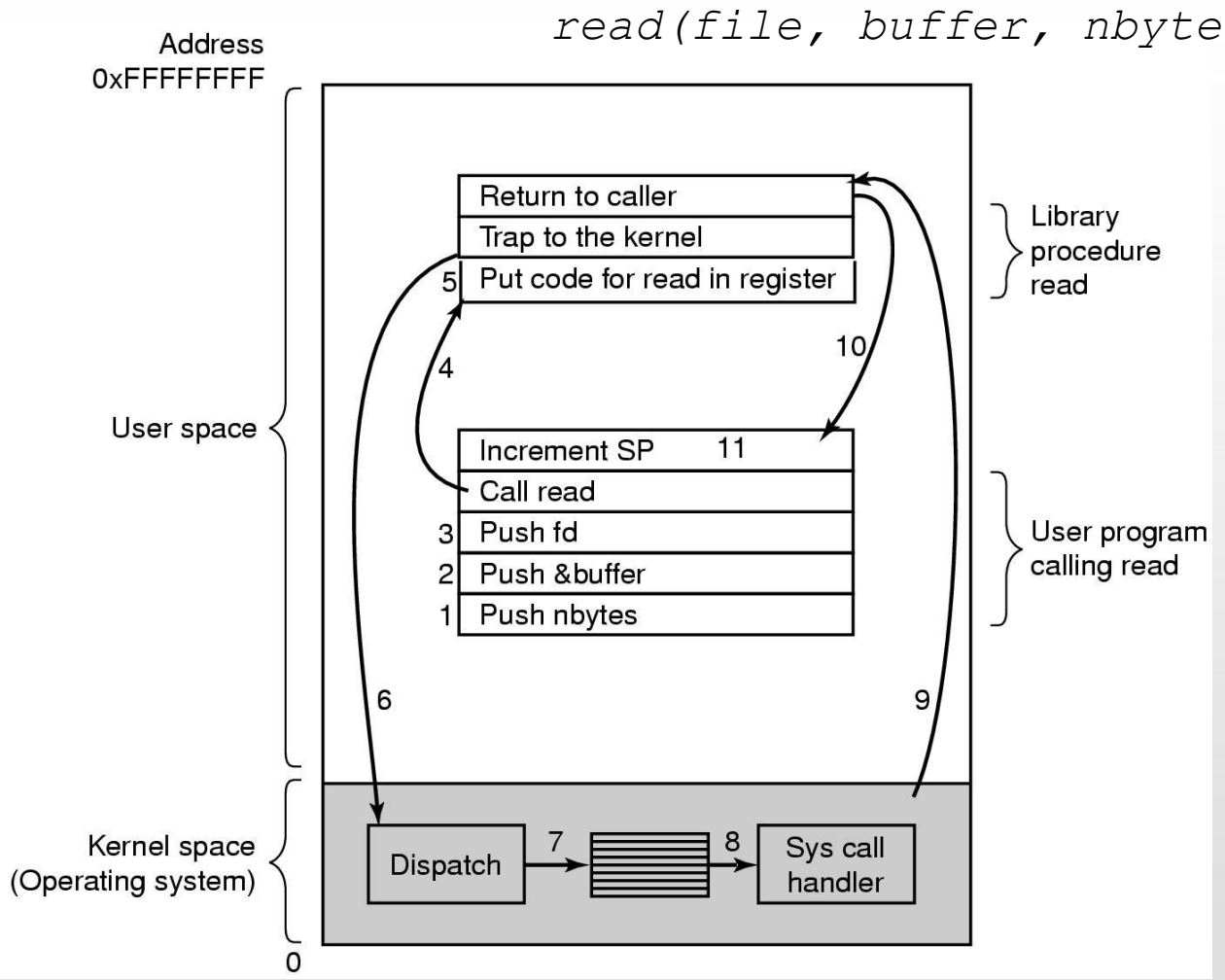
System Calls

- Es la forma en que los programas de usuario acceden a los servicios del SO.
- Los parámetros asociados a las llamadas pueden pasarse de varias maneras: por registros, bloques o tablas en memoria ó la pila.

```
count=read(file, buffer, nbytes);
```

- Se ejecutan en modo kernel o supervisor

System calls (cont.)



System Calls - Categorias

- Categorías de system calls:
  - ✔ Control de Procesos
  - ✔ Manejo de archivos
  - ✔ Manejo de dispositivos
  - ✔ Mantenimiento de información del sistema
  - ✔ Comunicaciones

System calls - Categorias (cont.)

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

System calls - Categorias (cont.)

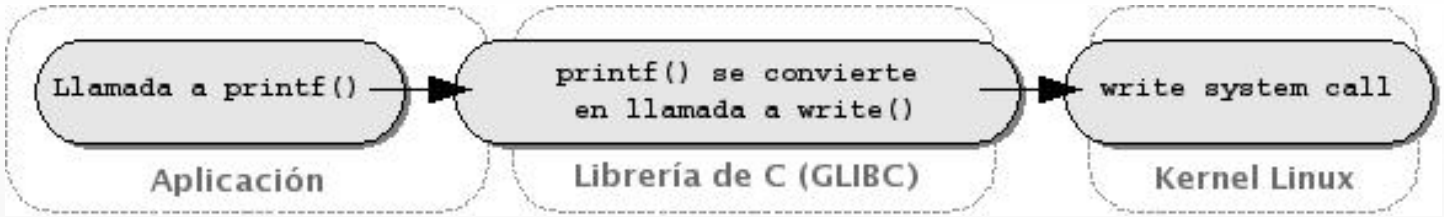
Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

Ejemplo - System Call Linux



- Para activar iniciar la system call se indica:
  - el número de syscall que se quiere ejecutar
  - los parámetros de esa syscall
- Luego se emite un aviso al SO (trap/excepción) para pasar a modo Kernel y gestionar la system call
- Se evalúa la system call deseada y se ejecuta

Systems Calls - Ejemplos

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time